

Reinforcement Learning

Unit -6

- reinforcement learning, which is a branch of machine learning that deals with learning via interaction and feedback.

Deep Reinforcement Learning Masters

Atari Games

- DeepMind astonished the machine learning community by unveiling a deep neural network that could learn to play Atari games with superhuman skill.
- This network, termed a *Deep Q-Network (DQN)* was the first large-scale successful application of reinforcement learning with deep neural networks.
- DQN was so remarkable because the same architecture, without any changes, was capable of learning 49 different Atari games, despite each game having different rules, goals, and gameplay structure.

What Is Reinforcement Learning?

- Reinforcement learning, at its essentials, is learning by interacting with an environment.
- This learning process involves an *actor*, an *environment*, and a *reward signal*.
- The actor chooses to take an action in the environment, for which the actor is rewarded accordingly.
- The way in which an actor chooses actions is called a *policy*. The actor wants to increase the reward it receives, and so must learn an optimal policy for interacting with the environment

Reinforcement architecture



- supervised learning, we are given data and labels, and are tasked with predicting labels given data.
- In unsupervised learning, data and are tasked with discovering underlying structure in this data.
- In reinforcement learning, we are given neither data nor labels.
- Our learning signal is derived from the rewards given to the agent by the environment.
- Reinforcement learning is exciting to many in the artificial intelligence community
- because it is a general-purpose framework for creating intelligent agents.
- Given an environment and some rewards, the agent learns to interact with that environment to maximize its total reward. This type of learning is more in line with how humans develop.
- Furthermore, reinforcement learning applications are at the forefront of many cutting-edge technologies including self-driving cars, robotic motor control, game playing, air-conditioning control, ad-placement optimization, and stock market trading strategies.

Markov Decision Processes (MDP)

- Our pole-balancing example has a few important elements, which we formalize as a
- *Markov Decision Process (MDP). These elements are:*
- **State**
- The cart has a range of possible places on the x-plane where it can be. Similarly,
- the pole has a range of possible angles.
- **Action**
- The agent can take action by moving the cart either left or right.
- **State Transition**
- When the agent acts, the environment changes—the cart moves and the pole
- changes angle and velocity.
- **Reward**
- If an agent balances the pole well, it receives a positive reward. If the pole falls,
- the agent receives a negative reward.

MDP

- An MDP is defined as the following:
- S , a finite set of possible states
- A , a finite set of actions
- P, r, s', s, a , a state transition function
- R , reward function

Mathematical framework for modeling decision-making

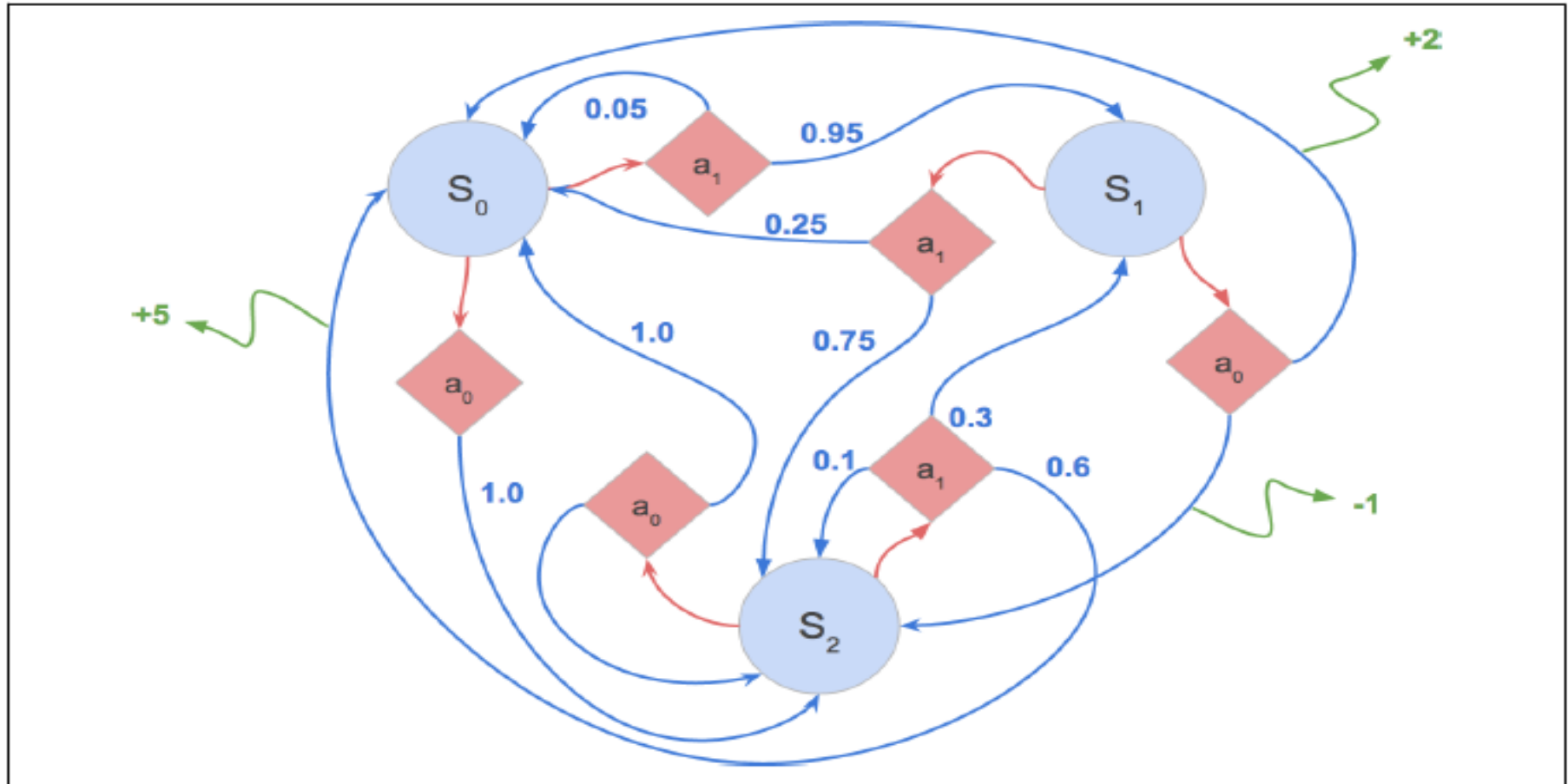


Figure 9-4. An example of an MDP. Blue circles represent the states of the environment. Red diamonds represent actions that can be taken. The edges from diamonds to circles represent the transition from one state to the next. The numbers along these edges represent the probability of taking a certain action. The numbers at the end of the green arrows represent the reward given to the agent for making the given transition.

- As an agent takes action in an MDP framework, it forms an *episode*. An *episode* consists of series of tuples of states, actions, and rewards.
- Episodes run until the environment reaches a terminal state, like the “Game Over” screen in Atari games, or when the pole hits the ground in our pole-cart example.
- The following equation shows the variables in an episode:

$$(s_0, a_0, r_0), (s_1, a_1, r_1), \dots (s_n, a_n, r_n)$$

- In pole-cart, our environment state can be a tuple of the position of the cart and the angle of the pole, like so:

$$(x_{cart}, \vartheta_{pole}).$$

Policy

- MDP's aim is to find an optimal policy for our agent. *Policies are the way in which*
- our agent acts based on its current state. Formally, policies can be represented as a
- function π *that chooses the action a that the agent will take in state s .*
- The objective of our MDP is to find a policy to maximize the expected future return:

$$\max_{\pi} E[R_0 + R_1 + \dots R_t | \pi]$$

Explore Versus Exploit

- Reinforcement learning is fundamentally a trial-and-error process.
- In such a framework, an agent afraid to make mistakes can prove to be highly problematic.
- A mouse is placed in the maze
- Our agent must control the mouse to maximize reward. If the mouse gets the water, it receives a reward of +1; if the mouse reaches a poison container (red), it receives a reward of -10; if the mouse gets the cheese, it receives a reward of +100.
- Upon receiving reward, the episode is over.
- The optimal policy involves the mouse successfully navigating to the cheese and eating it.

 +100				
 -10				
 -10		 -10		
		 +1		

- the mouse takes the left route, steps on a trap, and receives a -10 reward.
- In the second episode, the mouse avoids the left path, since it resulted in such a negative reward, and drinks the water immediately to its right for a +1 reward. After two episodes, it would seem that the mouse has found a good policy. It continues to follow its learned policy on subsequent episodes and achieves the moderate +1 reward reliably. Since our agent utilizes a greedy strategy—always choosing the model's best action—it is stuck in a policy that is a *local maximum*.
- To prevent such a situation, it may be useful for the agent to deviate from the model's recommendation and take a suboptimal action in order to *explore more of the environment*.
- So instead of taking the immediate right turn to *exploit the environment* to get water and the reliable +1 reward, our agent may choose to take a left turn and venture into more treacherous areas in search of a more optimal policy.
- Too much exploration, and our agent fails to optimize any reward. Not enough exploration can result in our agent getting stuck in local minimum. This balance of *explore versus exploit is crucial to learning a successful policy*.

Q-Learning and Deep Q-Networks

- Q-learning is in the category of reinforcement learning called value-learning.
- Instead of directly learning a policy, we will be learning the value of states and actions.
- Qlearning involves learning a function, a *Q-function*, *which represents the quality of a state, action pair*. The Q-function, defined $Q(s, a)$, *is a function that calculates the maximum discounted future return when action a is performed in state s*.
- The Q-value represents our expected long-term rewards, given we are at a state, and take an action, and then take every subsequent action perfectly (to maximize expected future reward). This can be expressed formally as:

$$Q^*(s_t, a_t) = \max_{\pi} E \left[\sum_{i=t}^T \gamma^i r^i \right]$$

Improving and Moving Beyond DQN

- DQN's many weaknesses include that it takes very long to train, doesn't work well on certain types of games, and requires retraining for every new game.

Deep Recurrent Q-Networks (DRQN)

- DRQN uses a recurrent layer to transfer a latent knowledge of state from one time step to the next.
- the model itself can learn how many frames are informative to include in its state and can even learn to throw away non-informative ones or remember things from long ago.
- DRQN has even been extended to include neural attention mechanism
- Since DRQN is dealing with sequences of data, it can attend to certain parts of the sequence.
- This ability to attend to certain parts of the image both improves performance and provides model interpretability by producing a rationale for the action taken.
- DRQN has shown to be better than DQN at playing first-person shooter (FPS) games like DOOM,⁶ as well as improving performance on certain Atari games with long time-dependencies, like Seaquest.

Asynchronous Advantage Actor-Critic Agent (A3C)

- *Asynchronous advantage actor-critic (A3C) is a new approach to deep reinforcement learning*
- A3C is asynchronous, which means we can parallelize our agent across many threads, which means orders of magnitude faster training by speeding up our environment simulation.
- A3C runs many environments at once to gather experiences.
- Beyond the speed increase, this approach presents another significant advantage in that it further decorrelates the experiences in our batches, because the batch is being filled with the experiences of numerous agents in different scenarios simultaneously.

A3C

- A3C uses an *actor-critic* method.
- *Actor-critic methods involve learning both a value function V (the critic) and also a policy π (the actor).*
- We delineated two different approaches to reinforcement learning: value learning and policy learning.
- A3C combines the strengths of each, using the critic's value function to improve the actor's policy.
- A3C uses an *advantage function instead of a pure discounted future return*.
- *When doing policy learning, we want to penalize the agent when it chooses an action that*

UNsupervised REinforcement and Auxiliary Learning (UNREAL)

- *UNREAL is an improvement on A3C introduced in “Reinforcement learning with*
- *unsupervised auxiliary tasks”*
- UNREAL addresses the problem of reward sparsity.
- Reinforcement learning is so difficult because our agent just receives rewards, and it is hard to determine exactly why rewards increase or decrease, which makes learning difficult. Additionally, in reinforcement learning, we must learn a good representation of the world as well as a good policy to achieve reward.
- Doing all of this with a weak learning signal like sparse rewards is quite a tall order.
- UNREAL asks the question, what can we learn from the world without rewards, and
- aims to learn a useful world representation in an unsupervised matter.
- Specifically, UNREAL adds some additional unsupervised auxiliary tasks to its overall objective.
- The first task involves the UNREAL agent learning about how its actions affect the
- environment.

- The agent is tasked with controlling pixel values on the screen by taking actions.
- To produce a set of pixel values in the next frame, the agent must take a specific action in this frame.
- In this way, the agent learns how its actions affect the world around it, enabling it to learn a representation of the world that takes into account its own actions.
- The second task involves the UNREAL agent learning *reward prediction*.
- *Given a* sequence of states, the agent is tasked with predicting the value of the next reward received.
- UNREAL is able to learn around 10 times faster than A3C on the Labyrinth game environment.
- UNREAL highlights the importance of learning good world representations and how unsupervised learning can aid in weak learning signal or low-resource learning problems like reinforcement learning.